

Tecnología Digital VI — Segundo Parcial

Modelo de Examen 4

Universidad Torcuato Di Tella

Duración: 2 horas

Nombre: _____

Parte 1: Verdadero o Falso (2 pts c/u)

Justifique brevemente las respuestas falsas.

- a) Una CNN tiene significativamente menos parámetros que un MLP para procesar imágenes, gracias al *parameter sharing* y la localidad de los filtros.
 - b) La fórmula del tamaño de salida de una convolución es $O = \frac{I-K+2P}{S} + 1$. Si aplicamos un kernel de 5×5 con padding $P = 2$ y stride $S = 1$ a una imagen de 28×28 , la salida tiene tamaño 28×28 .
 - c) Los Saliency Maps calculan la derivada de la predicción respecto a los pesos de la red para determinar qué píxeles son más importantes.
 - d) En las GANs, el Generador y el Discriminador se entrenan cooperativamente para generar imágenes realistas.
 - e) TF-IDF es una mejora sobre Bag-of-Words que penaliza palabras muy frecuentes en todo el corpus (como artículos y preposiciones), dando más peso a palabras discriminativas.
 - f) En un Decoder-Only Transformer como GPT, se usa *causal masking* para que cada token solo pueda atender a los tokens anteriores (y a sí mismo).
-

Parte 2: Multiple Choice (3 pts c/u)

Marque la opción correcta. Solo una es válida.

- 1) ¿Cuál es la principal contribución de la arquitectura **VGG**?
 - A) Demostró que usar siempre filtros de 3×3 con stride 1 es más eficiente que filtros grandes.
 - B) Introdujo las skip connections para mitigar vanishing gradients.
 - C) Fue la primera CNN en usar Dropout.
 - D) Redujo los parámetros usando Global Average Pooling.

- 2) ¿Qué es *teacher forcing* en el entrenamiento de un modelo Seq2Seq?
- A) Usar un modelo pre-entrenado (“teacher”) para inicializar los pesos del Seq2Seq.
 - B) En cada paso del decoder, usar como input la palabra correcta del ground truth en vez de la predicción anterior.
 - C) Forzar al encoder a producir un hidden state de mayor dimensión.
 - D) Aplicar regularización L2 a los pesos del decoder.
- 3) ¿Cuál de las siguientes analogías vectoriales es correcta usando embeddings de Word2Vec?
- A) $v_{\text{madrid}} \approx v_{\text{paris}} - v_{\text{francia}} + v_{\text{españa}}$
 - B) $v_{\text{madrid}} \approx v_{\text{paris}} + v_{\text{francia}} - v_{\text{españa}}$
 - C) $v_{\text{madrid}} \approx v_{\text{paris}} + v_{\text{francia}} + v_{\text{españa}}$
 - D) $v_{\text{madrid}} \approx v_{\text{españa}} - v_{\text{paris}} - v_{\text{francia}}$
- 4) ¿Qué scheduler reduce el learning rate a la mitad cuando la loss de validación deja de mejorar durante varias épocas?
- A) Step Decay
 - B) Exponential Decay
 - C) Cosine Annealing
 - D) Reduce on Plateau
- 5) En un Autoencoder, la **representación latente** es:
- A) La salida final reconstruida del decoder.
 - B) El vector comprimido que se obtiene entre el encoder y el decoder.
 - C) Los pesos de la red entrenada.
 - D) El vector de entrada después del preprocesamiento.

Parte 3: Desarrollo (10 pts c/u)

- 1) **Convolución: cálculo paso a paso.**

Dada una imagen de entrada de 6×6 (1 canal) y un kernel de 3×3 :

$$\text{Imagen} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix} \quad \text{Kernel} = \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

- a) ¿Cuál es el tamaño de la salida si se usa padding $P = 0$ y stride $S = 1$?
- b) Calcule el valor de la posición (0,0) del feature map de salida (esquina superior izquierda).

c) ¿Qué tipo de patrón detecta este kernel en particular?

2) Interpretabilidad: Grad-CAM vs Saliency Maps.

- a) Explique cómo funcionan los Saliency Maps: ¿respecto a qué se calcula la derivada y qué se mantiene fijo?
- b) ¿Qué mejora aporta Grad-CAM respecto a Saliency Maps? ¿En qué capa de la red opera?
- c) Describa brevemente cómo funcionan los ataques adversarios: ¿cómo se pueden modificar los píxeles de una imagen para engañar a la red?

3) LSTMs y dependencias a largo plazo.

- a) Explique por qué las RNNs simples sufren de vanishing gradients al entrenar con BPTT (Backpropagation Through Time).
- b) Describa la arquitectura interna de una celda LSTM: ¿qué son el cell state C_t y el hidden state h_t ? ¿Cuál es la función de cada gate (Forget, Input, Output)?
- c) ¿Por qué la operación de suma “+” en la actualización de C_t (en lugar de multiplicación) es clave para mitigar el vanishing gradient? Haga una analogía con las skip connections de ResNet.

Parte 4: Interpretación de Código (15 pts)

Analice el siguiente código de PyTorch y responda las preguntas.

```
1 import torch
2 import torch.nn as nn
3 import torchvision.models as models
4
5 # Cargar modelo pre-entrenado
6 resnet = models.resnet18(pretrained=True)
7
8 # Congelar todos los parametros
9 for param in resnet.parameters():
10     param.requires_grad = False
11
12 # Reemplazar la ultima capa FC
13 num_features = resnet.fc.in_features
14 resnet.fc = nn.Sequential(
15     nn.Linear(num_features, 256),
16     nn.ReLU(),
17     nn.Dropout(0.4),
18     nn.Linear(256, 2)
19 )
20
21 criterio = nn.CrossEntropyLoss()
22 # Solo optimizar los parametros de la nueva capa fc
```

```

23 optimizador = torch.optim.Adam(resnet.fc.parameters(), lr=0.001)
24
25 for epoch in range(20):
26     resnet.train()
27     for imgs, labels in train_loader:
28         optimizador.zero_grad()
29         outputs = resnet(imgs)
30         loss = criterio(outputs, labels)
31         loss.backward()
32         optimizador.step()

```

- a) ¿Qué técnica de Deep Learning se está aplicando en este código? Descríbala brevemente.
- b) ¿Qué efecto tiene `param.requires_grad = False` en el ciclo que recorre todos los parámetros? ¿Por qué se hace esto?
- c) ¿Por qué se pasa `resnet.fc.parameters()` al optimizador en vez de `resnet.parameters()`?
- d) La nueva capa `fc` tiene 2 neuronas de salida. ¿Cuántas clases tiene el problema? ¿Cuántas tenía el modelo original pre-entrenado en ImageNet?
- e) ¿Qué pasaría si las imágenes del `train_loader` fueran en escala de grises (1 canal) en vez de RGB (3 canales)?
- f) Si el modelo estuviera sobreajustando al nuevo dataset, ¿qué estrategia adicional (además del Dropout ya incluido) podría implementar sin cambiar la arquitectura? Mencione al menos dos opciones.